Time Is the Baseline to Beat: Evaluating VLA Failure Detectors Against Timers

Felix Doublet[*1] [TODO: co-authors]

[*1] [TODO: Institute of Science Tokyo, Shinoda Lab: confirm official affiliation]

Online failure detectors for vision-language-action (VLA) robot policies are usually compared by recall at a fixed false-positive rate (FPR). On RoboCasa365, a failed rollout is one that reaches its time limit, so a timer calibrated at 5% FPR flags all 380 prospective failures of Xiaomi-Robotics-1 on 38 tasks, and only warning lead can separate detectors. A per-task timer built from each task's successful durations nearly matches the SAFE score fused with a single timer. On fresh preregistered cohorts, SAFE adds 0.030 of the time limit to the per-task timer, below our declared margin of 0.05, while Hide-and-Seek, a sequence detector trained for early separation on the same features, adds 0.071 (95% CI 0.022–0.157), and 0.083 offline on a second VLA. Controls show that elapsed time explains how well detectors separate failures, but not this earlier warning. Detectors should be evaluated by their lead over a per-task timer.

1. Introduction

Generalist vision-language-action (VLA) policies still fail often on household tasks, so a deployed robot needs an online signal that the current attempt is going wrong. Recent detectors read the policy's hidden state and learn to predict eventual failure [1], or score how unusual that state is relative to successful rollouts [2, 3]. They are compared with each other, usually by ROC-AUC or by recall at a fixed false-positive rate (FPR), but rarely with what the robot knows for free: how long the attempt has been running.

That comparison matters because of how failure is defined. In RoboCasa365, as in most simulated manipulation benchmarks, an episode ends at first success or at a time limit. In our prospective test cohort, every one of the 380 failed rollouts ran exactly to its task's limit. "Detecting" such a failure therefore means predicting that the limit will be reached, and a timer that alarms at 90% of the limit already does so for all 380 failures at 4.7% FPR. Completed-rollout duration alone reaches ROC-AUC 0.80 on held-out rollouts, against 0.53 for the raw SAFE score. SAFE removes this shortcut offline by truncating rollouts to equal length [1], but a deployed detector sees rollouts of unequal length, and a timer is a legitimate deployable detector.

Against the timer, a learned detector can add only one thing: an earlier alarm at the same false-alarm rate, which leaves more time to stop, reset or ask for help. This paper is an evaluation, not a new detector: we test learned detectors, SAFE [1] and then Hide-and-Seek [4], against a single timer and a per-task timer. Detectors are developed on one cohort, their thresholds are frozen on a separate calibration cohort at 5% FPR, and they are evaluated once on a disjoint prospective cohort; the main comparison is then repeated on a fresh, preregistered cohort. Our contributions are:

1. **The timer baseline.** A calibrated timer flags every failure within the FPR budget, while a survival-style time-risk curve misses 37%, and we show why.
2. **Lead time as the endpoint.** We measure the mean warning lead, the fraction of the time limit left when the first alarm fires (zero for a missed failure).
3. **How much the policy adds.** A per-task timer that never reads the policy nearly matches SAFE fused with a single timer. On fresh preregistered cohorts, SAFE adds 0.030 of the limit to the per-task timer, below our declared margin of 0.05, while Hide-and-Seek adds 0.071. Controls show that elapsed time explains how well detectors separate failures, but not this earlier warning.

Contact: Felix Doublet, [TODO: affiliation, postal address, e-mail]

2. Related Work

Failure detectors for VLA policies. SAFE trains a small network on a VLA's last-layer features to predict eventual failure and thresholds it with a functional conformal band [1]; we reuse its implementation. Follow-ups change the training data, labels or features [4–8]. FAIL-Detect [2] and FIPER [3] score the policy against successful rollouts only; FIPER alarms when two scores *both* cross, whereas we alarm when *either* does. Action-chunk monitors [9–11] compare overlapping chunks, which our policy does not produce. Check-VLA [12], the closest work on our benchmark, verifies each chunk with a world model, and Foresight [13] uses world-model latents; vision-language monitors judge progress from frames [14–17]. None of these works compares against elapsed time. Duration is a shortcut feature [18], but its causal form is legitimate evidence online.

Time and intervention. Our time risk is a survival-style estimate [19, 20], and lead time at a fixed false-alarm budget is the accuracy–earliness trade-off of early classification [21]. RESCUE [22] shows that good detection scores do not guarantee lower intervention cost, so the value of an earlier alarm must eventually be shown in closed loop.

3. Problem Setting

Time. We measure time in four units. The simulator advances in *control steps* at 20 Hz, RoboCasa's default control rate, which our collection code does not change. The policy is queried once per 16-step action chunk and executes the whole chunk, so one *policy inference* covers 16 control steps, or 0.8 s. Inferences are our clock: $t = 1, 2, \ldots$ counts them, and a detector can change its decision only at an inference. The official time limit of task k is T_k control steps; in inferences it is $H_k = \lceil T_k/16 \rceil$, between 29 and 272 for our tasks (22.5 to 217.5 s). Every fraction of time in this paper, such as t/H_k, the timer fraction c and the lead ℓ, is a fraction of H_k.

Monitoring. A detector watches one rollout of a known task k and decides at each inference t whether to alarm, using only the policy features so far, t, the task and H_k; it may not use the rollout's final length or outcome, and it never changes

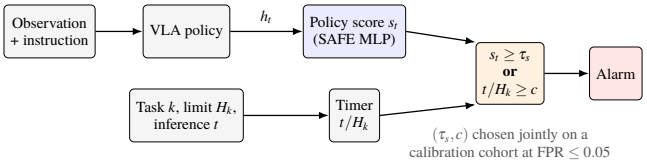


Figure 1: SAFE + timer. At each policy inference, the policy score and the elapsed fraction of the time limit are compared with two thresholds; the first crossing of either raises the alarm. The timer guarantees an alarm on every failure; the policy score can only move it earlier.

the policy's actions. The first threshold crossing is the alarm T (in inferences). Failure is the positive class. A successful rollout with any alarm before it ends is a false positive, and FPR is the fraction of successful rollouts with a false alarm. An alarm at any inference up to and including the last one counts, with $\ell = 0$ at the last one. For a failed rollout, which ends at H_k, the *warning lead* is $\ell = (H_k - T)/H_k$ if an alarm fires and $\ell = 0$ otherwise. We report the mean lead $\bar{\ell}$ over failures, which counts a miss as no warning and so combines recall and earliness in one number. Unlike recall, it does not saturate when a timer catches every failure, and unlike a median over detected failures, it cannot be improved by dropping hard failures. A timer at fraction c has $\bar{\ell} \approx 1 - c$ by construction, up to rounding to whole inferences, so any gain over it must come from alarms before $c\,H_k$. We also report recall, FPR, the median lead over detected failures and the fraction alarmed by $0.75\,H_k$. Detectors are compared at operating points fixed on a calibration cohort under FPR $\leq \alpha = 0.05$. Confidence intervals come from a paired bootstrap that resamples tasks, then rollouts within each task (2,000 replicates). In the exploratory analysis, only the SAFE + timer versus timer comparison was singled out in advance; the confirmatory tests have their own registered decision rules (Sec. 7). All other intervals are descriptive.

4. Detectors

Timers. The global timer alarms once $t \geq c\,H_k$ for a fraction $c \in (0, 1]$ shared by all tasks; it uses only the task's time limit. The per-task timer uses $c_k = \min(1, \rho_k + \delta)$, where ρ_k is the 95th percentile of task k's development success durations as a fraction of H_k. The shared margin δ is set on the calibration successes by a split-conformal quantile rule [23], which bounds the FPR on a new successful rollout by 0.05. It needs successful rollouts of each task, but no failures and no policy features.

Time risk. The survival-style alternative estimates

$$r_t = \Pr(\text{failure} \mid \text{task } k,\ \text{still running at } t) \tag{1}$$

from development rollouts. With n_t of them still running at t and f_t of those failing, it fits an increasing isotonic regression in t/H_k to $(f_t + 0.5)/(n_t + 1)$, weighted by n_t. Because every failure runs to H_k, r_t is in effect a task-specific timer whose threshold is shared across tasks.

Policy score. At each inference we take the final-layer action-token hidden state of the policy's flow-matching action head (1,024 dimensions). The SAFE MLP [1] maps it to a failure score $u_t^{(j)}$ for each of three training seeds j. With per-task location $m_k^{(j)}$ and scale $\sigma_k^{(j)}$ estimated on development rollouts, the policy score is

$$s_t = \frac{1}{3} \sum_{j=1}^{3} \frac{\max_{t' \leq t} u_{t'}^{(j)} - m_k^{(j)}}{\sigma_k^{(j)}}. \tag{2}$$

The running maximum makes s_t monotone and does not change when a threshold is first crossed. We use the MLP head because it ranked failures best after task normalization on held-out development data (ROC-AUC 0.69 against 0.65 for the LSTM head). We also fit four success-only scorers on the same hidden state (Mahalanobis [24], cosine kNN [25], PCA-KMeans and RND [26]), using successful development rollouts only.

Sequence score and controls. Hide-and-Seek [4] trains an LSTM on a 128-dimensional PCA of the same hidden state with rollout labels only, using a margin loss that pushes each failure's peak score above every success's; its scores are normalized like SAFE's. Because a recurrent model can learn how long each task takes, we retrain it unchanged on zero inputs (it sees only elapsed time), on the features of a random same-task training rollout at the same inference (also the task's typical trajectory), and on the rollout's own first-inference features repeated (also the episode's initial appearance).

OR-fusion. A fused detector alarms at the first inference where either signal crosses its threshold, for example

$$T = \min\{\, t : s_t \geq \tau_s \text{ or } t \geq c\,H_k \,\} \tag{3}$$

for SAFE + timer (Fig. 1). The pair of thresholds is searched jointly over the per-rollout maxima of the calibration cohort; among pairs with calibration FPR ≤ 0.05, we keep the one with the largest mean lead, or the highest recall for the original SAFE + time-risk design. This selection is empirical: choosing the best of many pairs on 114 calibration successes is optimistic and carries no finite-sample FPR guarantee. SAFE + per-task timer instead moves both thresholds along one path ($\delta = \lambda$, $\tau_s = u + s\lambda$, with u and s chosen on calibration failures) and sets λ by the same conformal rule.

Null fusions. To test whether a fusion gains only because failed rollouts run longer and give any score more chances to cross, we fuse the timer with the time risk (a second time signal) and with Gaussian noise (the running maximum of i.i.d. draws, 20 draws), and calibrate them exactly like SAFE + timer.

5. Experimental Setup

Policy and tasks. We use the released Xiaomi-Robotics-1 RoboCasa365 checkpoint [27]. It predicts 16-step action chunks with a five-step flow-matching action head and executes each chunk in full. We take the 39 official RoboCasa365 tasks [28, 29] whose published success count is between 5 and 45 of 50, and drop one with too few failures, leaving 38 tasks (10 atomic, 28 composite). Failed rollouts last 29 to 272 inferences (median 127, about 100 s).

Cohorts and protocol. Five disjoint cohorts each serve one purpose (Table 1). Each evaluation cohort is collected from fresh environment seeds until every task has a fixed quota of successes and failures, and the primary cohort takes the first rollouts of each class per task in seed order.

Table 1: Cohorts. S/F = successes/failures; q = rollouts per class per task. No two cohorts share an environment seed.

Cohort	Used to	Primary (S/F, q)
Development	fit SAFE, scorers, time risk	1,824 rollouts
Calibration	set thresholds	228 (114/114, 3)
Prospective test	evaluate once	760 (380/380, 10)
Confirm. calibration	thresholds for Sec. 7	760 (380/380, 10)
Confirmatory	preregistered test	760 (380/380, 10)

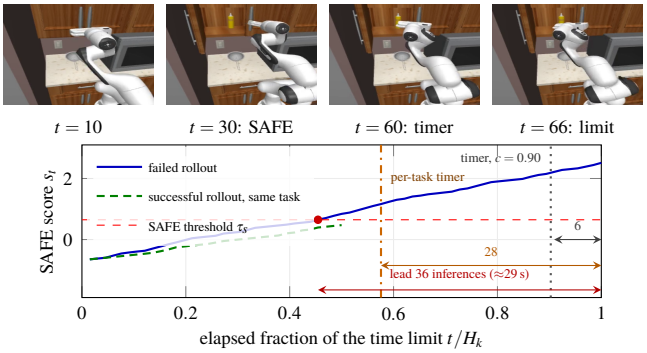


Figure 2: One failed *OpenCabinet* rollout from the prospective test (chosen to illustrate; the median SAFE-first lead is smaller). The SAFE score crosses its calibrated threshold at $t = 30$ of $H_k = 66$ inferences, 36 inferences before the time limit; the timer alarms at $t = 60$, and the per-task timer, which never reads the policy, at $t = 38$. Dashed: the longest of the 9 (out of 10) successful test rollouts of this task that stay below the threshold; it ends at $t = 33$.

SAFE, the normalizations, the time-risk curves and an original staged detector were frozen before the prospective test was collected. The timer and all fused detectors were added afterwards and evaluated once on the opened cohort, with thresholds from calibration only; each analysis first reproduces the frozen results and checks the input checksums. The results in Sec. 6 are therefore exploratory; Sec. 7 reports the preregistered confirmatory test.

6. Exploratory Results

The timer is the baseline to beat. With $c = 0.903$ chosen on calibration (FPR 0.044 there), the timer alarms on all 380 test failures at 0.047 test FPR (Table 2). Its lead is fixed by construction: 0.090 of the limit, a mean of 11.7 inferences or about 9 s. Against the time-risk curve, SAFE and the four success-only scorers raised recall from 0.632 to 0.90–0.96; against the timer, those gains vanish.

Why the time risk misses. All 140 misses come from the 14 tasks with at most 14 failed development rollouts: with f of them, the curve rises only to $(f+0.5)/(f+1)$, below the shared threshold $\tau_r = 0.972$ unless $f \geq 17$. Smoothing and a shared threshold, not elapsed time, cause the misses.

SAFE adds lead over a single timer. SAFE + timer keeps recall at 1.000 and raises the mean lead to 0.207 of the limit, a paired gain of 0.117 over the timer (95% CI 0.060–0.178), or 20.2 against 11.7 inferences (about 16 s against 9 s). The SAFE threshold fires first on 194 failures (51%), and 42% of failures are flagged by $0.75\,H_k$, against none for the timer; Fig. 2 shows one such rollout. Calibration chose a late timer fallback ($c = 0.96$) and spent the false-alarm budget on the SAFE threshold, so the median lead over detected failures (10 inferences) is below the timer's: the gain is concentrated in the failures SAFE flags early, and the remaining failures are flagged later than by the timer alone.

Table 2: Prospective test (380/380), thresholds from calibration at FPR ≤ 0.05. $\bar{\ell}$: mean lead (miss = 0); Med: median lead (inferences) of detected failures; @.75: fraction alarmed by $0.75\,H_k$. *Mean of 20 draws ($\bar{\ell}$ 0.088–0.134). †Conformal margin on the 114 calibration successes.

Detector	Recall	FPR	$\bar{\ell}$	Med	@.75
Timer ($c = 0.903$)	**1.000**	0.047	0.090	11.5	0.000
Time risk	0.632	0.047	0.120	16.5	0.211
Time risk + timer	**1.000**	0.050	0.125	11.0	0.105
Noise + timer*	**1.000**	0.066	0.101	–	–
SAFE only	0.566	0.039	0.190	23.0	0.421
SAFE + time risk	0.955	0.058	0.184	12.0	0.311
SAFE + timer	**1.000**	0.058	**0.207**	10.0	0.421
Per-task timer†	**1.000**	0.045	0.195	13.5	0.342
SAFE + per-task timer†	**1.000**	0.042	0.202	14.0	0.389

A per-task timer removes the gain. The per-task timer reaches $\bar{\ell} = 0.195$ at 0.045 FPR without reading the policy. SAFE + timer leads it by an insignificant 0.012 (CI -0.044 to 0.063), and by -0.003 against a per-task timer tuned on test to the same FPR. SAFE + per-task timer reaches 0.202, a gain of 0.007 (CI -0.003 to 0.019), far below the 0.05 of the limit we would call useful. The two effects cancel across tasks: where the time risk never fires, SAFE + timer leads the per-task timer (0.405 against 0.221); elsewhere the per-task timer leads (0.180 against 0.092). In Fig. 2, the per-task timer alarms at $t = 38$, eight inferences after SAFE.

A sequence detector does warn earlier. Fused with the per-task timer by the same conformal rule, Hide-and-Seek gains 0.098 over it (CI 0.037–0.158), and 0.083 (0.022–0.143) against the per-task timer loosened to its own 21 test false alarms; SAFE gains 0.008 in that matched comparison, and the best of 100 noise draws 0.016. The zero-input control matches Hide-and-Seek's ROC-AUC (0.888 against 0.878), yet adds 0.001 over the per-task timer at matched FPR, and the same-task shuffle -0.004: elapsed time explains separation, not earliness. ActProbe, the other supervised baseline we tried, gains 0.009.

Time alone and noise do not reproduce it. Fusing the time risk with the timer, which uses every time signal we have, reaches $\bar{\ell} = 0.125$. Noise fused with the timer under the same search reaches 0.101 at a higher FPR (0.066). SAFE also separates outcomes among rollouts of equal length: among rollouts still running at 10, 25, 50 and 75% of the limit, its within-task ROC-AUC is 0.55–0.60, against 0.41–0.51 for noise and exactly 0.5 for any time signal.

Atomic and composite tasks. The lead gain is largest on the 10 atomic tasks (SAFE + timer 0.364 against 0.079 for the timer, at 0.090 FPR) and smaller on the 28 composite tasks (0.151 against 0.094); per-type FPR is not controlled, so this is descriptive.

Table 3: Confirmatory test (380/380). Gain against the per-task timer P. Rows from P down: registered decision bootstrap; above P: descriptive. *Mean (maximum) of 20 draws.

Detector	FPR	$\bar{\ell}$	Gain vs P
Global timer	0.024	0.123	−0.102 (−0.147, −0.059)
SAFE + time risk (frozen)	0.039	0.190	−0.035 (−0.066, −0.007)
SAFE + global timer	0.047	0.215	−0.011 (−0.068, 0.045)
Per-task timer P	0.055	0.225	–
Noise / resampled SAFE + P*	–	–	0.002 ($\leq$ 0.009)
SAFE + P	0.061	**0.255**	**0.030** (0.007, 0.093)
First-inference + P	0.055	0.226	0.001 (−0.015, 0.036)
Zero-input + P	0.061	0.237	0.012 (0.001, 0.044)
Shuffle + P	0.063	0.236	0.011 (0.000, 0.064)
Hide-and-Seek + P	0.061	**0.296**	**0.071** (0.022, 0.157)

7. Confirmatory Tests

Protocol. A fresh cohort of 760 rollouts (new seeds, 10 per class per task) was collected with SAFE frozen, first for a recall endpoint. Before any timer or lead was computed on it, we registered an amendment: the per-task timer P (95th percentile) is the comparator, the mean lead gain $\Delta\bar{\ell}$ of SAFE + P over P is the endpoint, and the gain counts as useful if its 95% CI lies above zero, it is at least 0.05 of the limit, and SAFE + P's test FPR is at most 0.075; the cohort's recall and FPR were already known. A new calibration cohort sets both thresholds with the conformal rule, and the decision interval resamples calibration and test together, refitting thresholds each time.

A small real gain, below our margin. Every test failure again ran to its limit, and P and SAFE + P caught all. SAFE + P leads P by 0.030 of the limit (Table 3), with a CI above zero and a test FPR of 0.061, but the gain is below the declared 0.05, so it does not count as useful. Noise and within-task resampled SAFE scores fused with P gain at most 0.009, so the gain is policy evidence. Its CI reaches 0.093, so a useful gain is not ruled out, and the opened-cohort estimate (0.007) understated it. Part of it is FPR budget: a per-task timer tuned on test to SAFE + P's FPR leaves 0.015 (CI −0.007 to 0.041). Task duration carries most of SAFE's lead: P leads the global timer by 0.102, and the original SAFE + time-risk design falls 0.035 behind P. The four success-only scorers fused with P gain 0.020–0.034 (exploratory), and atomic tasks carry most of every score's gain.

A second amendment: Hide-and-Seek passes. After the exploratory result, we registered the same rules for Hide-and-Seek and its three controls, before any of them was scored on these cohorts; the retrained detector reproduced its exploratory scores exactly. Hide-and-Seek + P leads P by 0.071 (CI 0.022–0.157) at 0.061 FPR (Wilson bound 0.089), meeting every rule, and noise or resampled scores gain at most 0.012. The controls gain 0.001–0.012. Hide-and-Seek beats the first-inference control by 0.070 (CI 0.014–0.153), so its warning develops during the rollout rather than reflecting a hard-looking scene, and beats the zero-input and shuffle controls by about 0.06 (CIs above zero). Its advantage over SAFE + P, 0.041 (CI −0.011 to 0.103), is not significant.

8. Discussion and Limitations

On benchmarks where failure means reaching a time limit, a calibrated timer already detects every failure, so a learned VLA failure detector should be credited only for warning earlier than a timer at the same false-alarm rate. Against a per-task timer built from successful durations, SAFE adds 0.030 of the limit on a fresh preregistered cohort: real, but below what we declared useful. Elapsed time explains how well detectors separate failures: an LSTM that sees only time ranks failures as well as one that reads the policy. It does not explain all early warning: Hide-and-Seek, trained for early separation on SAFE's features, adds 0.071, and that warning develops during the rollout. The lead-over-timer evaluation reveals this difference; recall does not. Video labels of 48 confirmatory failures, made blind to alarms, suggest why (exploratory): 79% were still recoverable at the limit, mostly slow rather than irreversibly wrong (second-labeler agreement on 12 videos: Cohen's $\kappa = 0.80$). The confound is not specific to RoboCasa (exploratory, on others' released data): in FIPER's rollouts [3], length alone separates failures with ROC-AUC 0.92–1.00 in four of five tasks. In FIPER's own code, an added elapsed-time method has the best balanced accuracy (0.81 against 0.78) at test FPR 0.09 against 0.35, yet at FIPER's FPR, FIPER warns 0.15 of the episode earlier than the timer (post hoc), so the lead test credits FIPER. Where every rollout runs to a fixed horizon, as in SAFE's real-robot set, the timer is uninformative: the termination rule decides. Offline, on a second policy (RLDX-1, 100 test failures), all failures also run to the limit; SAFE's LSTM head + P gains 0.034 (CI 0.008–0.067) and Hide-and-Seek + P 0.083 (0.033–0.138; 0.052 at matched FPR), both only on atomic tasks.

For benchmark design: when every failure is a timeout, recall rewards any score that grows with time, so papers should report lead over a per-task timer, and benchmarks should label failures that become unrecoverable early.

Limitations. (i) The timer and all fused detectors were defined after the prospective cohort had been opened, and the confirmatory endpoint was registered after that cohort's recall and FPR were known (but before its lead). (ii) The two-threshold search is optimistic (test FPR 0.050–0.066 against 0.044 on calibration), and even the conformal rule let SAFE + P and Hide-and-Seek + P run above P's FPR (0.061 against 0.055). (iii) The gain is a dozen inferences on average, mostly late in the rollout (no detector flags over 8% of failures by half the limit), and whether an earlier alarm raises task success through reset and retry is untested. (iv) The simulator ends failed episodes only at the time limit, so failures that become unrecoverable earlier (drops, collisions) are not labelled. (v) SAFE needs white-box access and per-task development failures; the timer needs only the time limit. (vi) All results are simulated, with known task and time limit; one policy is tested prospectively, a second (RLDX-1) only offline, on existing data with a small test set. (vii) The per-task timer's percentile was chosen after the exploratory cohort was opened. (viii) Hide-and-Seek was the better of two supervised baselines on the opened cohort, and was registered after SAFE's confirmatory result, on cohorts already opened for SAFE.

9. Conclusion

Elapsed time is the baseline to beat: on fresh data, SAFE added 0.030 to a per-task timer, Hide-and-Seek 0.071.

References

[1] Qiao Gu, Yuanliang Ju, Shengxiang Sun, Igor Gilitschenski, Haruki Nishimura, Masha Itkina, and Florian Shkurti. SAFE: Multitask failure detection for vision-language-action models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[2] Chen Xu, Tony Khuong Nguyen, Emma Dixon, Christopher Rodriguez, Patrick Miller, Robert Lee, Paarth Shah, Rares Ambrus, Haruki Nishimura, and Masha Itkina. Can we detect failures without failure data? Uncertainty-aware runtime failure detection for imitation learning policies. In *Robotics: Science and Systems (RSS)*, 2025.

[3] Ralf Römer, Adrian Kobras, Luca Worbis, and Angela P. Schoellig. Failure prediction at runtime for generative robot policies. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[4] Seongheon Park, Wendi Li, Changdae Oh, Min-Hsuan Yeh, Zsolt Kira, Michael Hagenow, and Sharon Li. Hide-and-seek in trajectories: Discovering failure signals for VLA runtime monitoring. In *ICML Workshop on Failure Modes in Agentic AI*, 2026. arXiv:2605.30834.

[5] Harshitha Rajaprakash, Aditeya Prajapati, Rong Xue, Abrar Anwar, and Jesse Thomason. SAFECAST: Robust failure detection for VLA policies with contrast-set training and calibration, 2026. arXiv:2608.04246.

[6] Jie Ma, Zongxi Liu, and Yi Zhu. FailureSpot: Label-efficient timestamp-level failure detection for vision-language-action models, 2026. arXiv:2609.04277.

[7] Jinghan Yang, Yunchao Zhang, Wang Yuan, Haolun Wan, Jiaming Zhang, Zhengyang Hu, and Yanchao Yang. Tri-Info: Generalizable, interpretable failure prediction for VLA models via information theory, 2026. arXiv:2606.19998.

[8] Kaiwen Zhu, Dongfang Liu, and Liangkai Liu. VLA-Scope: Shift-aware failure prediction for vision-language-action models, 2026. arXiv:2609.21246.

[9] Christopher Agia, Rohan Sinha, Jingyun Yang, Zi-ang Cao, Rika Antonova, Marco Pavone, and Jeannette Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *Conference on Robot Learning (CoRL)*, volume 270 of *PMLR*, 2024.

[10] Florian Seligmann, Emiliyan Gospodinov, Enes Ulas Dincer, and Gerhard Neumann. VLA-FAIL: Efficient task failure detection for fine-tuned vision-language-action models, 2026. arXiv:2606.21386.

[11] Bingjia Huang, Xiangyu Li, Xiang Wang, Liang Mi, Zixu Hao, Weijun Wang, Hao Wu, Kun Li, Yunxin Liu, and Ting Cao. ActProbe: Action-space probe for early failure detection of generative robot policies, 2026. arXiv:2606.08508.

[12] Yushan Liu, Peibo Sun, Xintao Chao, Zhenyang Yang, Yifan Xie, Lingfeng Zhang, Shoujie Li, Chenyu Tang, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. CheckVLA: Execution-time verification with action-conditioned world model for long-horizon mobile manipulation, 2026. arXiv:2607.26789.

[13] Boyang Wang, Haoran Zhang, Mengdi Wang, Odest Chadwicke Jenkins, Xuhui Kang, Yen-Ling Kuo, Yifu Lu, and Zezhou Cheng. Foresight: Failure detection for long-horizon robotic manipulation with action-conditioned world model latents, 2026. arXiv:2606.23085.

[14] Zeyi Liu, Arpit Bahety, and Shuran Song. REFLECT: Summarizing robot experiences for failure explanation and correction. In *Conference on Robot Learning (CoRL)*, 2023.

[15] Rohan Sinha, Amine Elhafsi, Christopher Agia, Matthew Foutter, Edward Schmerling, and Marco Pavone. Real-time anomaly detection and reactive planning with large language models. In *Robotics: Science and Systems (RSS)*, 2024. doi: 10.15607/RSS.2024.XX.114.

[16] Enshen Zhou, Qi Su, Cheng Chi, Zhizheng Zhang, Zhongyuan Wang, Tiejun Huang, Lu Sheng, and He Wang. Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6919–6929, 2025.

[17] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2025.

[18] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.

[19] Edward L. Kaplan and Paul Meier. Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282):457–481, 1958.

[20] David R. Cox. Regression models and life-tables. *Journal of the Royal Statistical Society: Series B*, 34(2):187–220, 1972.

[21] Zhengzheng Xing, Jian Pei, and Philip S. Yu. Early classification on time series. *Knowledge and Information Systems*, 31(1):105–127, 2012. doi: 10.1007/s10115-011-0400-x.

[22] David Ahmedt-Aristizabal, Abdelwahed Khamis, Mohammad Ali Armin, and Lars Petersson. RESCUE: Is this alarm worth acting on? Counterfactual evaluation of VLA runtime monitors, 2026. arXiv:2610.13516.

[23] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[24] Kimin Lee, Kibok Lee, Honglak Lee, and Jinwoo Shin. A simple unified framework for detecting out-of-distribution samples and adversarial attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[25] Yiyou Sun, Yifei Ming, Xiaojin Zhu, and Yixuan Li. Out-of-distribution detection with deep nearest neighbors. In *International Conference on Machine Learning (ICML)*, 2022.

[26] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations (ICLR)*, 2019.

[27] Xiaomi Robotics Team. Xiaomi-Robotics-1: Scaling vision-language-action models with over 100K hours of real-world trajectories, 2026. arXiv:2607.15330.

[28] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. RoboCasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.

[29] Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots. In *International Conference on Learning Representations (ICLR)*, 2026.